

Documentação Teórica da Construção

Por que cada técnica é a escolha certa — fundamentação conceitual

Projeto: MWRPG — A Coroa Enterrada de Ys

Versão: 1.0

Autoria: Claude (Anthropic), sob coordenação de Tiago Bocchino

Data: 2026-08-31

Sumário

1. Introdução	3
2. Por que 2d6, e não 1d20	3
3. Por que o contrato do mestre é JSON estrito, não texto livre	3
4. Por que magic link primeiro, senha depois	3
5. Por que retrieval por tag é suficiente hoje, e quando deixaria de ser	4
6. Por que o limite de demo é por campanha, não por usuário nem por sala	4
7. Por que CRS.Simple para um mapa que não é geográfico	4
8. Por que licença de conteúdo importa antes do código, não depois	5
9. Referências	5

1. Introdução

O Documento 03 cataloga **o que** foi aplicado; este documento explica **por que** cada técnica é a escolha correta para o problema que resolve — a fundamentação teórica por trás de cada mecanismo central do MWRPG.

2. Por que 2d6, e não 1d20

A soma de dois dados de 6 lados não é uniforme como um único d20 — é uma **distribuição triangular**: os resultados centrais (6, 7, 8) são muito mais prováveis que os extremos (2 ou 12). Isso é uma escolha de design deliberada, não um detalhe técnico neutro: um sistema baseado em 1d20 dá a mesma chance (5%) pra cada resultado, então "falha catastrófica" e "sucesso pleno" são igualmente prováveis a cada rolagem — ótimo pra tensão de combate tático, ruim pra um jogo cujo pilar é "foco em narrativa, com sistema de regras leve" (CLAUDE.md §2). Com 2d6, o resultado mais comum cai exatamente na banda "sucesso parcial (7-9)" — o sistema estruturalmente favorece "você consegue, mas com custo" como o desfecho típico, que é o motor narrativo mais rico pro Mestre IA trabalhar (sempre tem uma complicação pra narrar, raramente um beco sem saída total).

3. Por que o contrato do mestre é JSON estrito, não texto livre

Pedir a um LLM que narre e **também** decida dados estruturados (quantas opções oferecer, se a cena mudou de local, se HP mudou) tem duas soluções possíveis: parsear texto livre depois com regex/heurística, ou forçar o próprio modelo a devolver JSON. A segunda é estruturalmente mais robusta porque desloca o trabalho de extração pro momento em que o modelo já "sabe" a estrutura da própria resposta — o mesmo princípio por trás de *structured output/function calling* em APIs de LLM modernas, aplicado aqui via instrução de prompt (a Groq usada (openai/gpt-oss-120b) não expõe um parâmetro de JSON mode formal verificado; a garantia vem do prompt mais `temperature: 0.9` e do fato de que o campo é sempre validado no cliente antes de renderizar).

O ganho estrutural mais importante: o contrato é **independente do provedor**. Trocar de Claude pra Groq (Documento 07, Seção 2) não tocou em uma linha do frontend — só no adaptador de request/response — porque `narration/mode/options/mapHint/stateChanges` são um contrato de dados, não uma convenção de texto que cada modelo interpretaria diferente.

4. Por que magic link primeiro, senha depois

A ordem de atrito de um funil de cadastro é teoria de UX bem estabelecida: cada campo extra pedido antes do primeiro valor entregue ao usuário reduz conversão. Um formulário de senha na primeira tela (nome + senha + confirmação) pede compromisso antes de o jogador ter visto o produto; um magic link pede só o email — o menor compromisso possível pra provar identidade sem senha nenhuma pra lembrar.

A senha entra **depois**, na confirmação do link (`SetPasswordGate`, opcional/pulável) — nesse ponto o jogador já demonstrou intenção real (clicou o link, voltou pro jogo), então o custo de pedir

mais um passo é menor, e o ganho é real: evita depender de um novo link mágico (que exige abrir o email de novo) toda vez que o jogador retorna. É uma aplicação prática do princípio "peça o compromisso maior só depois do menor compromisso ser recompensado" — mas é também, como o Documento 07 (Seção 4, retrospectiva) registra com honestidade, uma decisão que nunca foi formalmente contestada por um Frontend Engineer consultado — ela funciona, mas o processo que deveria tê-la escrutinado não rodou.

5. Por que retrieval por tag é suficiente hoje, e quando deixaria de ser

`pickAcervoLore()` (Documento 03, Seção 5) usa `indexOf` simples sobre 6 entradas — não busca vetorial, sem embeddings. Isso é **teoricamente correto pro tamanho atual**: com N pequeno (aqui, $N=6$), o custo de uma busca de similaridade semântica (calcular embedding da consulta, comparar contra embeddings pré-calculados, rankear por distância de cosseno) tem overhead maior que o problema que resolve — uma varredura linear por sobreposição de palavra-chave já encontra o subconjunto relevante quase sempre, porque a chance de colisão acidental de tag é baixa com poucas entradas.

Essa técnica **deixaria de ser suficiente** no momento em que o acervo crescesse o bastante pra (a) ter entradas com tags parecidas mas sentido distinto (colisão semântica que um `indexOf` não distingue), ou (b) o volume tornar a varredura linear perceptível em latência — nenhum dos dois é o caso hoje (o roadmap v0.5 original, RAG com Supabase pgvector, é justamente o ponto de virada planejado, ainda não justificado pelo volume atual do acervo).

6. Por que o limite de demo é por campanha, não por usuário nem por sala

Três unidades possíveis de limite existiam: por usuário (conta), por sala (grupo), ou por campanha (uma instância de jogo). A escolha por campanha tem uma justificativa dupla:

1. **Convite, não punição**: um limite por usuário soa como banimento permanente depois de uma campanha ("você já usou sua cota"); por campanha soa como "esta história específica dura até aqui" — convida a começar de novo, não fecha a porta.
2. **Sustenta o custo compartilhado real**: o teto de tokens da Groq é por organização inteira (Documento 05, Seção 6) — o limite por campanha é a unidade que efetivamente protege esse orçamento compartilhado, já que é o que determina quanto uma sessão de jogo consome no total.

O modelo de dados atual (`campaign_sessions` 1:1 usuário, sem sala, Documento 02) faz "por campanha" coincidir com "por usuário" na prática — essa coincidência é consequência do escopo solo atual, não prova de que a escolha teórica estaria errada se sala existisse: nesse cenário, "por campanha" continuaria sendo a unidade certa, só que uma campanha passaria a pertencer a um grupo, não a uma conta individual.

7. Por que CRS.Simple para um mapa que não é geográfico

Leaflet foi desenhado originalmente pra mapas do mundo real (coordenadas de latitude/longitude, projeção Mercator). O MWRPG usa `L.CRS.Simple` — um sistema de coordenadas cartesiano puro, onde 1 unidade = 1 pixel da imagem, sem projeção geográfica nenhuma. Isso é teoricamente o uso correto da biblioteca fora do seu domínio nativo: Leaflet como *motor de navegação em plano* (zoom, pan, marcadores, tooltips) é geograficamente agnóstico por baixo da API de tile/projeção — `L.CRS.Simple` só troca a função de projeção por uma identidade. A conversão de coordenada usada (`L.latLng(imageHeight - pixelY, pixelX)`) existe porque o sistema de coordenadas de imagem tem origem no canto superior esquerdo (Y cresce pra baixo), enquanto o Leaflet, por herança do seu domínio geográfico original, espera Y crescendo pra cima (latitude) — a subtração inverte o eixo pra casar os dois sistemas.

8. Por que licença de conteúdo importa antes do código, não depois

Toda entrada do acervo narrativo (`docs/ACERVO-PROVENIENCIA.md`) e todo asset de mapa (`docs/MAPAS-PROVENIENCIA.md`) tem proveniência registrada **antes** de entrar em produção, não auditada depois — esse é o desenho correto porque o custo de remover conteúdo já publicado (e potencialmente já consumido/reproduzido por jogadores) é estruturalmente maior que o custo de verificar a fonte antes. A regra prática aplicada duas vezes no projeto — nunca copiar a prosa original de uma obra específica mesmo em domínio público (o mito é livre, a tradução/edição de um autor específico pode não ser) e nunca usar sprites de terreno de forma que produza um resultado visualmente diferente do que a licença CC0 do Kenney cobre — reflete o mesmo princípio: verificar a fonte resolve o problema de raiz; confiar na aparência de "parece genérico/livre" não resolve.

9. Referências

1. `src/engine.js` — distribuição de `roll2d6`, verificada por leitura direta.
2. `src/master.js` — `SYSTEM_PROMPT`, contrato JSON, lido integralmente.
3. `src/auth.js`, `src/components.jsx` (`LoginGate`, `SetPasswordGate`) — fluxo de atrito de cadastro.
4. `src/acervo.js`, `src/maps.js` — retrieval por tag e sistema de coordenadas Leaflet.
5. Leaflet. **Using a non-geographical CRS.**
<https://leafletjs.com/examples/crs-simple/crs-simple.html>
(<https://leafletjs.com/examples/crs-simple/crs-simple.html>). Documentação oficial do `CRS.Simple`.
6. `docs/ACERVO-PROVENIENCIA.md`, `docs/MAPAS-PROVENIENCIA.md` — disciplina de proveniência aplicada antes da publicação.